

Machine Learning for Natural Language Understanding

Prof. Achim Rettinger



Neural Nets

Lecture 03

Sources Lecture and Exercise

- Logistic Regression
 - <https://youtu.be/nUdiu96sWmE>
- Perceptron, Artificial Neural Networks, Backprop, Regularization
 - <https://youtu.be/QDX-1M5Nj7s?si=XWMX3af03WvX2M0b&t=888>

Flipped Classroom Questions

- Why is the quadratic loss not ideal for binary classification?
- How is the output of an artificial neuron (perceptron) calculated?
- What are the benefits of using several neurons in an artificial neural net?
- Name two options to reduce the risk of overfitting.

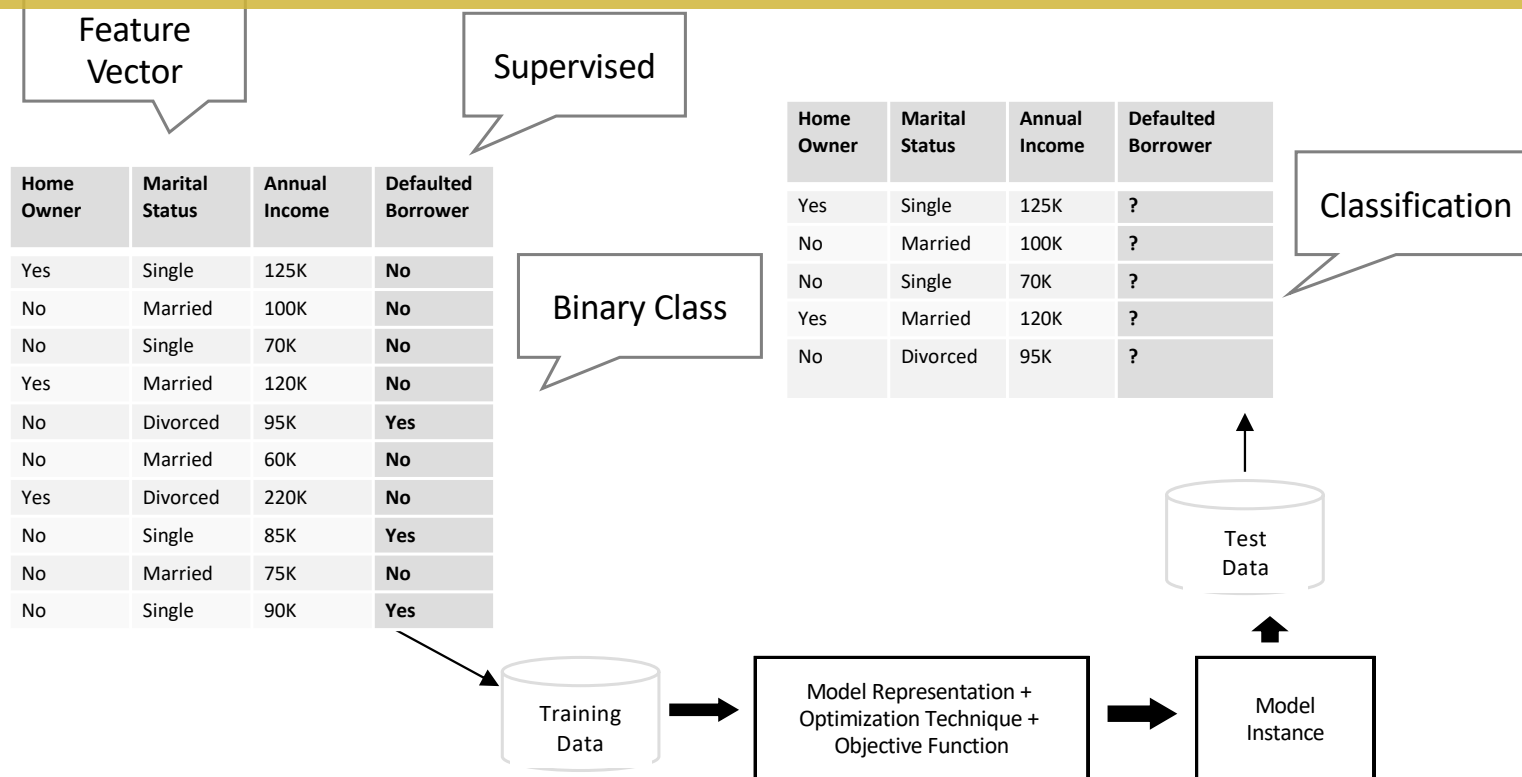
Logistic Regression

Chapter 3.1 - <https://youtu.be/nUdiu96sWmE>

Linear Classification (naive attempt)

Real-world Setting	Data Representation	Learning Paradigm	Task
	• Feature vector • Sequence • Graph •	• Supervised • Unsupervised • Reinforcement • ...	• Regression • Classification • Clustering • ...
Model Instantiation	Model Representation	Optimization Technique	Objective Function
	Linear Functions / Hyperplanes	Gradient Descent	Misclassification Rate

Recap: Binary Classification



Recap: Data Representation

Real-world
Setting

Data Representation	Learning Paradigm	Task
• Feature vector • Sequence • Graph •	• Supervised • Unsupervised • Reinforcement • ...	• Regression • Classification • Clustering • ...

Feature Vector				Supervised	
Home Owner	Marital Status	Annual Income	Defaulted Borrower	Binary Class	
Yes	Married	120K	No		
No	Divorced	95K	Yes		
No	Married	60K	No		

Training data:

$$\text{train} := \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\} \in \mathbb{R} \times \{0, 1\}$$

Classification model: $f(x) = \hat{y}$

Misclassification rate:

$$\frac{1}{|\text{test}|} \sum_{(x,y) \in \text{test}} 1 \text{ if } y \neq \hat{y}$$

Recap: Binary Classification with Linear Regression

Optimize a linear regression model

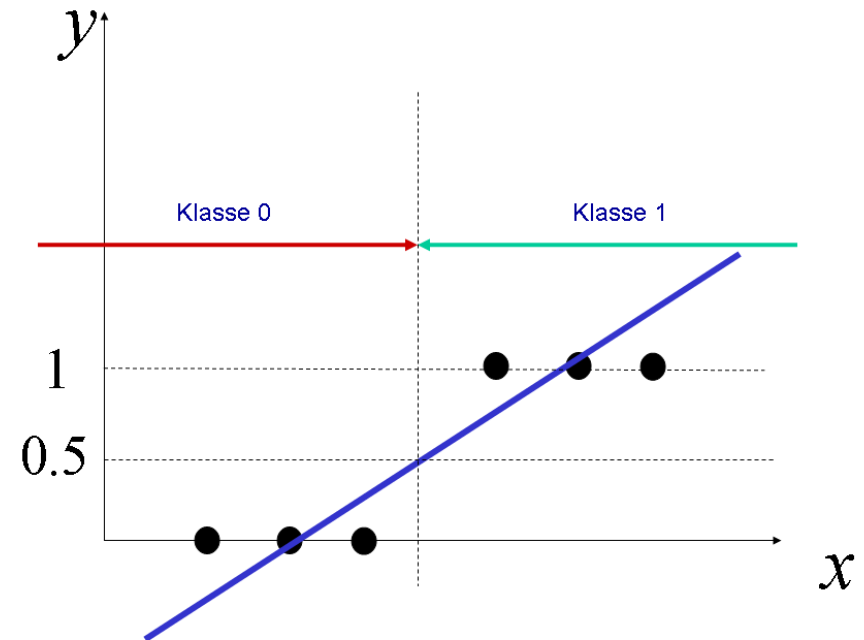
$$Y = \langle X, w \rangle$$

for Residual Sum of Squares (RSS) loss

$$\sum_{n=1}^N (y_i - x_i^T w)^2$$

Fundamental issue:

1. We predict something different than classes
2. Targets are not only 0 and 1, but unbounded



$$p(Y = 1|X, w)$$

Logistic Regression (proper attempt)

Real-world Setting	Data Representation	Learning Paradigm	Task
	• Feature vector • Sequence • Graph •	• Supervised • Unsupervised • Reinforcement • ...	• Regression • Classification • Clustering • ...
Model Instantiation	Model Representation	Optimization Technique	Objective Function
	Logistic Function $\text{logistic}(\langle X, w \rangle)$	Gradient Descent	Log Likelihood

Logistic / Sigmoid Function

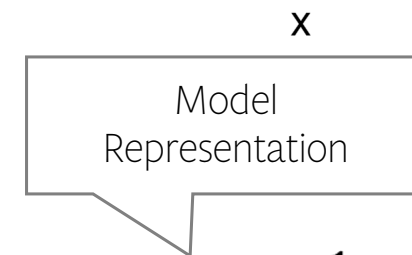
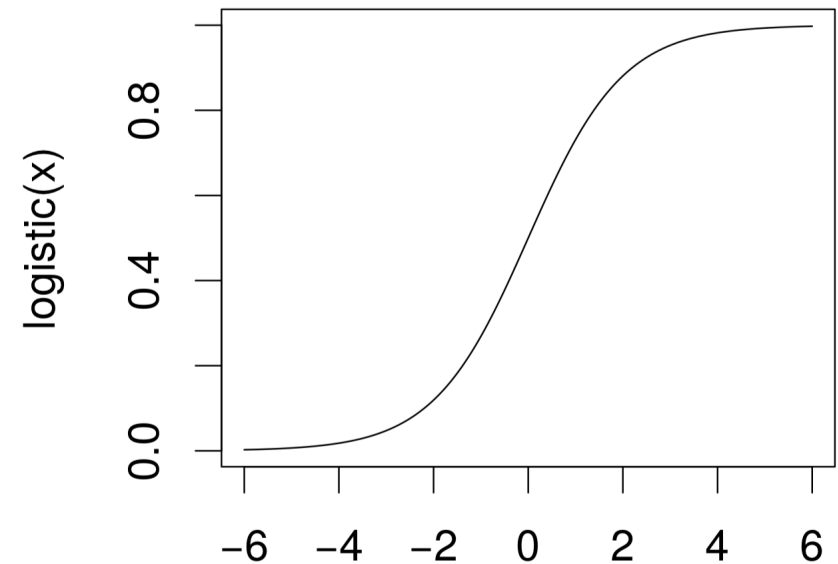
$$\text{logistic}(x) := \frac{1}{1 + e^{-x}}$$

Properties:

- Range $[0, 1]$
- Converges to 1 / 0
- Smooth and symmetric at $(0, 0.5)$

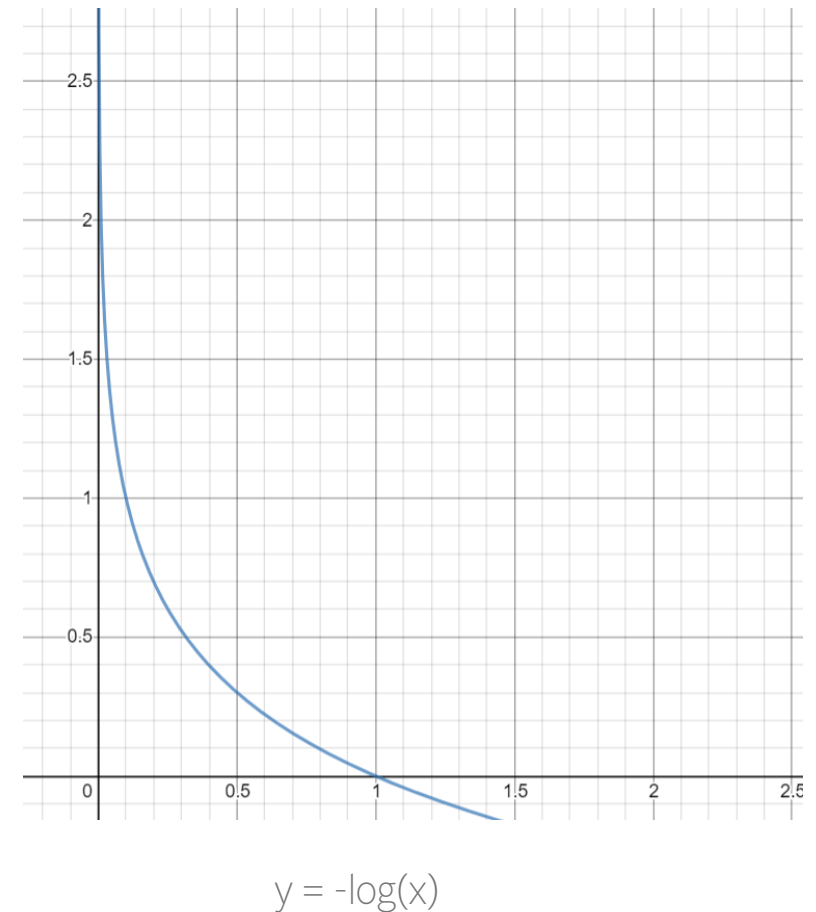
Logistic Regression Model:

$$p(Y = 1 | X, w) \approx \text{logistic}(\langle X, w \rangle) = \frac{1}{1 + e^{-\langle X, w \rangle}}$$



Log(arithmetic) Loss – How to calculate the error

- If the output p of the probabilistic classifier is the Probability of the input sample being positive (indicated by 1)
- Then, the error can be calculate using the negativ log likelihood $-\log(p)$
- Usually called **cross-entropy loss**



Loss over all test instances

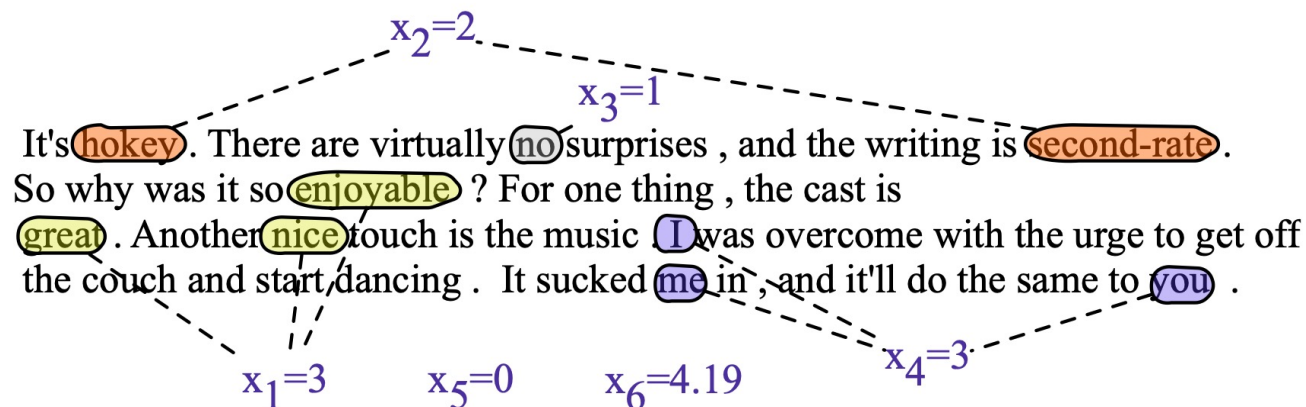
Misclassification rate:

$$\frac{1}{|\text{test}|} \sum_{(x,y) \in \text{test}} 1 \text{ if } y \neq \hat{y}$$

Continuous log loss:

$$\frac{1}{|\text{test}|} \sum_{(x,y) \in \text{test}} \begin{cases} -\log(p) & \text{if } (y = 1) \\ -\log(1 - p) & \text{if } (y = 0) \end{cases}$$

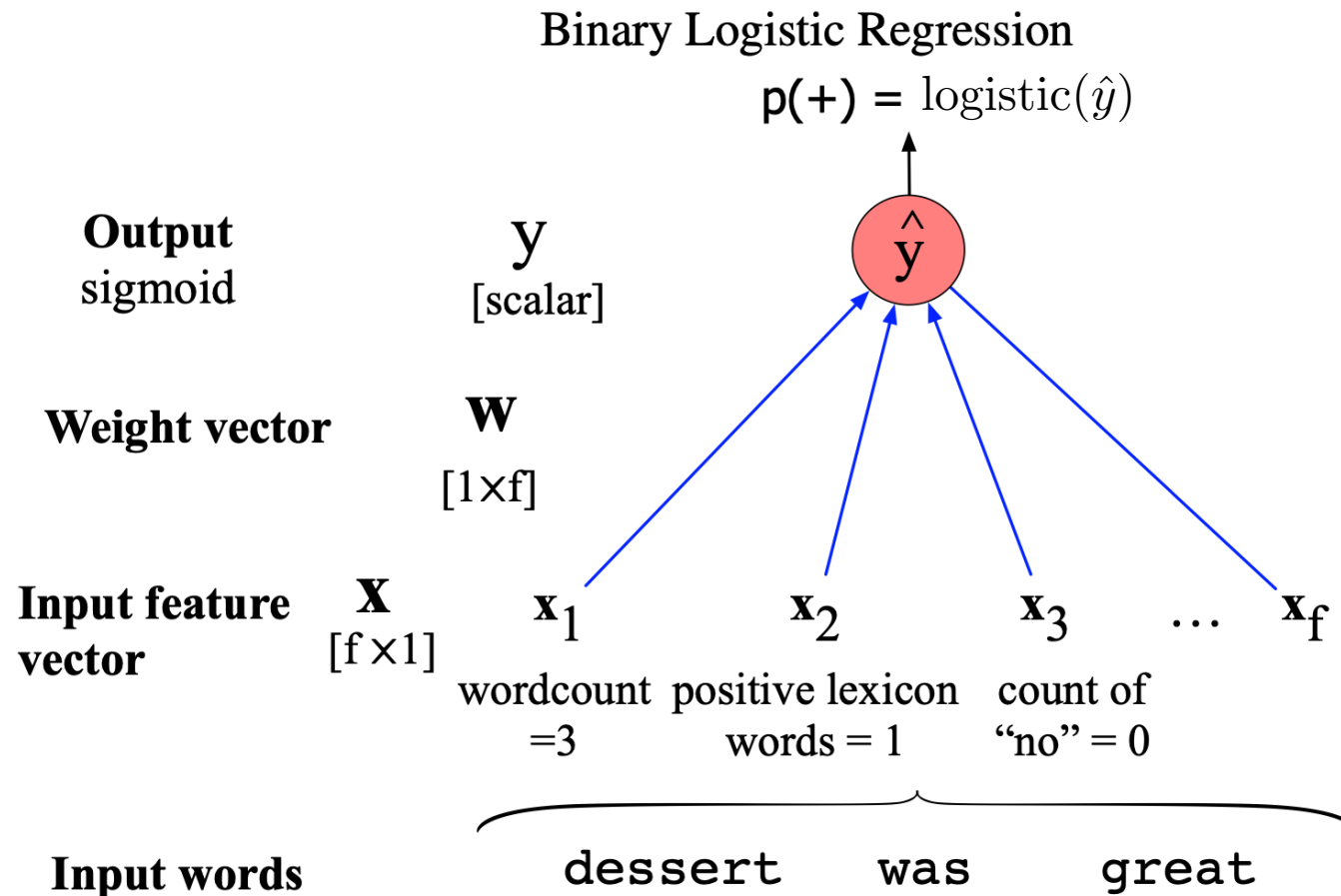
Example: Sentiment classification



Var	Definition
x_1	count(positive lexicon words $\in$ doc)
x_2	count(negative lexicon words $\in$ doc)
x_3	$\begin{cases} 1 & \text{if "no" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$
x_4	count(1st and 2nd pronouns $\in$ doc)
x_5	$\begin{cases} 1 & \text{if "!" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$
x_6	log(word count of doc)

<https://web.stanford.edu/~jurafsky/slp3/5.pdf>

Example: Sentiment classification



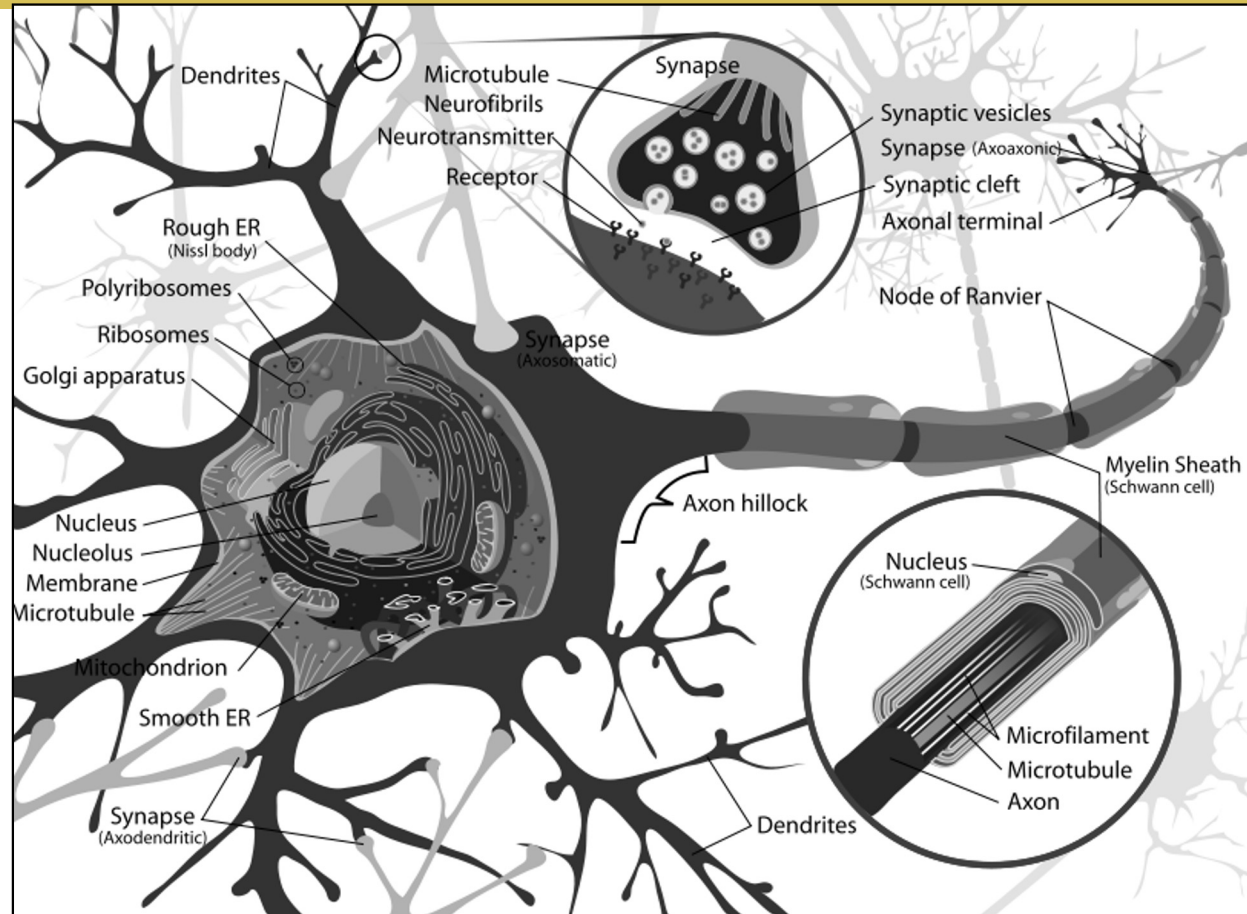
The Perceptron Algorithm

Chapter 3.2 - <https://youtu.be/QDX-1M5Nj7s?si=XWMX3af03WvX2M0b&t=888>

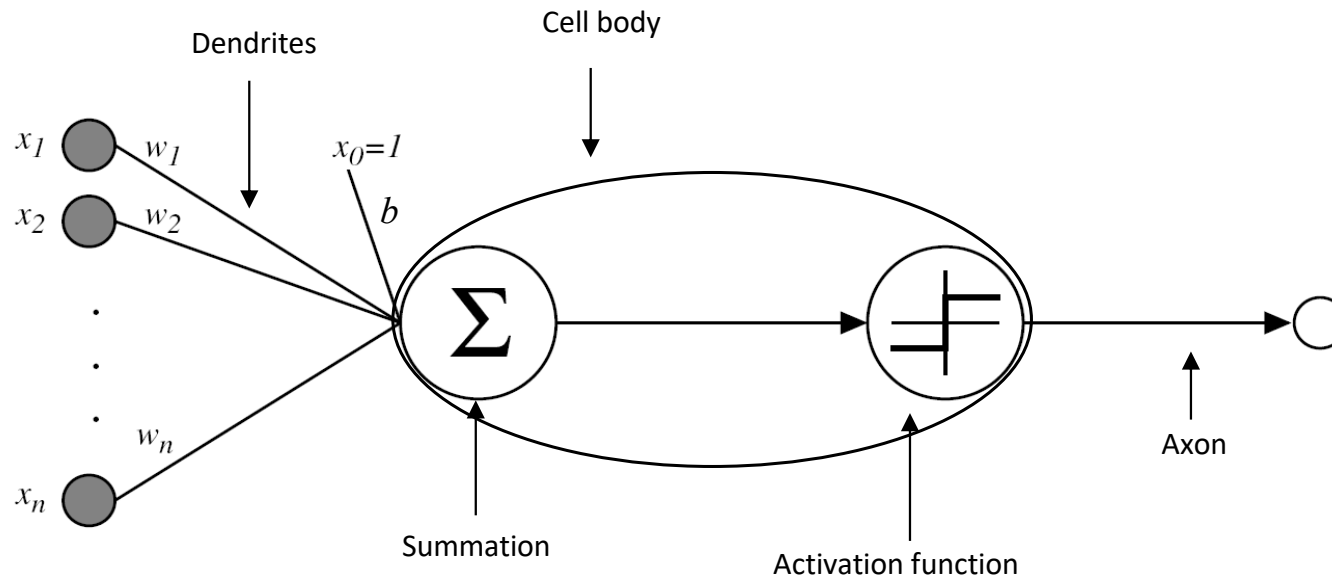
Introduction

- Artificial Neural Networks (ANNs) are learning systems inspired by the structure of neural information processing.
- Key ideas:
 - Simple, adaptive computational units (artificial neurons),
 - Connections between neurons for information propagation.
- Key properties:
 - ANNs are a powerful and flexible learning method,
 - ANNs show a black-box behavior: the eventual model is not easy to understand for humans and hard to analyze theoretically.

Biological Inspiration: Neural Information Processing



Abstract Model of the Neuron



Variants:

The McCulloch-Pitts Model (1947) considers only **binary inputs & weights**

McCulloch, W., and W. Pitts (1947), "How We Know Universals: the Perception of Auditory and Visual Forms", *Bulletin of Mathematical Biophysics*, Vol. 9, pp. 127–147.

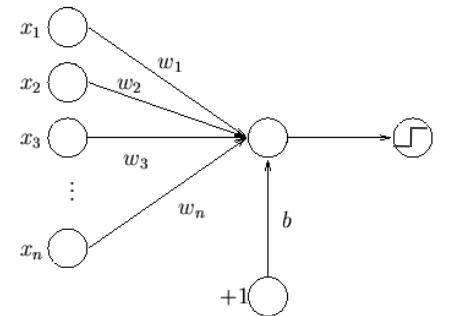
The Perceptron (Rosenblatt, 1958) considers any **real-valued inputs & weights**.

Rosenblatt, F. (1958), "The Perceptron: a Probabilistic Model for Information Storage and Organization in the Brain", *Psychological Review*, Vol. 65, pp. 386–408.

Formal Perceptron Model

Input Space X ; $X \subseteq \mathbb{R}^n$ (real vectors).

Output Domain $Y = \{-1, 1\}$ (yes/no).



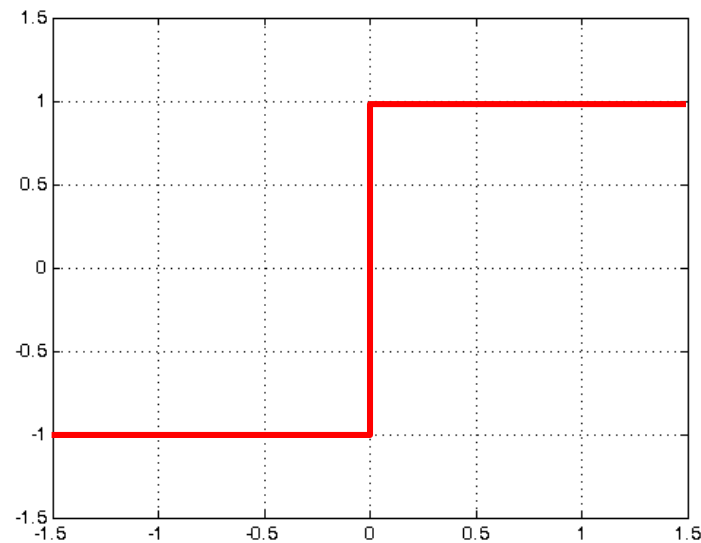
Perceptron Decision Function:

$$f(x_1 \dots x_n) = \begin{cases} 1 & \text{if } w_1x_1 + \dots + w_nx_n > \theta \\ -1 & \text{otherwise.} \end{cases}$$

or, using $\theta = -b$:

$$f(x_1 \dots x_n) = \begin{cases} 1 & \text{if } w_1x_1 + \dots + w_nx_n + b > 0 \\ -1 & \text{otherwise.} \end{cases}$$

Sign- (Threshold- / Step-) Function



- Convention: $\text{sign}(0) = -1$
- NB: alternative formulation for threshold $b = -\theta$ corresponds to step function at θ .

Perceptron: Linear Threshold Functions

- Perceptron encodes a simple **linear discriminant function**.
- Generic notation:

$$\hat{y} = f(x) = \text{sign}(\langle x, w \rangle + b) \quad (1)$$

$$= \text{sign} \left(\sum_{i=1}^n x_i w_i + b \right) \quad (2)$$

- NB: Many other learning algorithms (e.g. SVMs) use linear discriminant functions.
-
- (notation $n=M$ and $i=j$ in previous slides)

Perceptron Training Task (1st Version)

"Given a set of training instances, find parameters w and b that are capable of separating positive and negative input patterns in the training set."

NB: we will see that this is not always possible.

Revisit: “Perceptron Training” Algorithm

Algorithm 1 Perceptron Training.

Input: data $\mathcal{S}$; learning rate α

learning rate determines intensity of change (assumption: $\alpha=1$)

Initialize: $w \leftarrow 0, b \leftarrow 0$

```
1: repeat
2:    $err \leftarrow 0$ 
3:   for  $i = 1, \dots, \ell$  do
4:     compute  $f(x_i) = \text{sign}(\langle w, x_i \rangle + b)$ 
5:     if  $f(x_i) \neq y_i$  then
6:        $w \leftarrow w + \alpha y_i x_i$ 
7:        $b \leftarrow b + \alpha y_i$ 
8:        $err = err + 1$ 
9:     end if
10:  end for
11: until  $err = 0$ 
12: return  $w, b$ 
```

redirects weight vector towards/away from misclassified training point

adjusts bias for increased vector norm

Rosenblatt, F. (1958), “The Perceptron: a Probabilistic Model for Information Storage and Organization in the Brain”, Psychological Review, Vol. 65, pp. 386–408.

Linear Separability

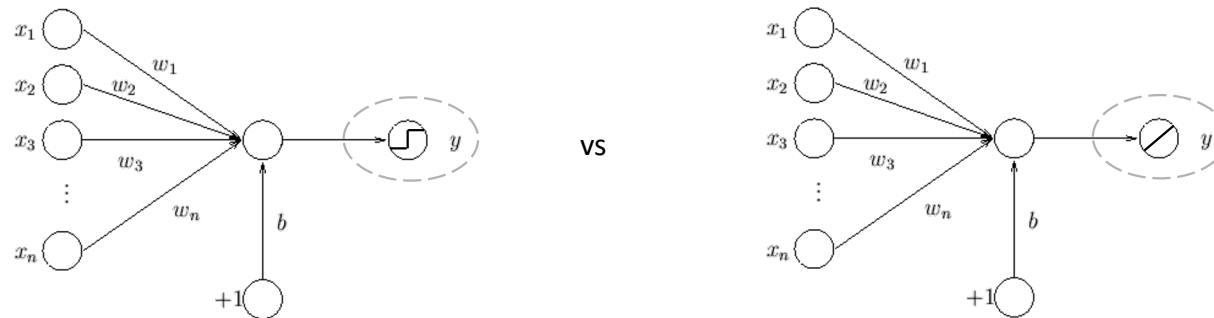
- **Linear Separability** – Classification problems for which a model can be expressed in the form of linear threshold functions are called **linearly separable**.
- Most real problems are, however, not linearly separable:
 - Case I: linear pattern exists but is blurred (noise) – we might be happy with a linear model but try to minimize the (unavoidable) error.
 - Case II: underlying pattern is not linear in principle – we will need a different (non-linear !) model.

Perceptron Training Task (2nd Version)

"Given a set of training instances, find parameters w and b that minimize the error on the training set."

Unthresholded Perceptron

- Consider for a while an unthresholded neuron (linear output unit):



- This corresponds to a linear regression model, i.e. a linear model for a numeric response variable.

$$\hat{y} = f(x) = \left(\sum_{i=1}^n x_i w_i + b \right)$$

Achim Rettinger

Recap: Gradient Descent Training for Perceptrons (Batch Version)

Algorithm 1 Gradient Descent (Batch)

Input: data $\mathcal{S}$; learning rate α

Initialize: $w \leftarrow 0, b \leftarrow 0$

1: **repeat**

2: $\Delta w \leftarrow 0, \Delta b \leftarrow 0$

3: **for** $i = 1, \dots, \ell$ **do**

4: compute $\hat{y}_i = \langle w, x_i \rangle + b$

5: $\Delta w \leftarrow \Delta w + \alpha (y_i - \hat{y}_i) x_i$

6: $\Delta b \leftarrow \Delta b + \alpha (y_i - \hat{y}_i)$

7: **end for**

8: $w \leftarrow w + \Delta w$

9: $b \leftarrow b + \Delta b$

10: **until** some termination criterion

11: **return** w, b

← imagine this to be a dummy feature that equals 1 for all inputs.

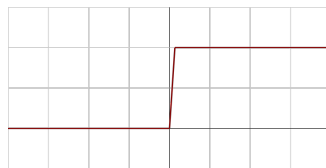
← w and b are updated after all examples have been processed

← see later

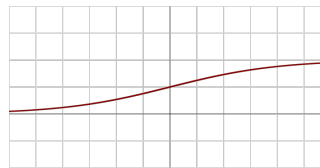
Choice of Activation Function, Learning Rate and Termination Criterion

- Learning rate α :
 - moderates the width of update steps, actually hard to choose
 - large steps may jump to far over a possible solution
 - small steps may lead to too many iterations
 - sometimes implemented as a function of the iterations which becomes smaller over time.
- Termination criterion (selection):
 - Error below certain threshold (This is a bad criterion! Why?)
 - Change of error between iterations below certain threshold.
- Activation Functions

- Binary step

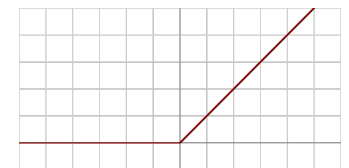


Sigmoid



https://en.wikipedia.org/wiki/Activation_function

ReLU



„Perceptron Training“ vs Gradient Descent

- „Perceptron Training“ is guaranteed to converge...
 - to feasible parameters
 - if training examples are linearly separable
- Gradient Descent / Delta Rule is guaranteed to converge approximately...
 - to parameters with minimum squared error E
 - even if training examples are not linearly separable
 - problem remains, however, if the training data is not separable in principle ("minimum" error will be very high).



Multilayer Perceptrons

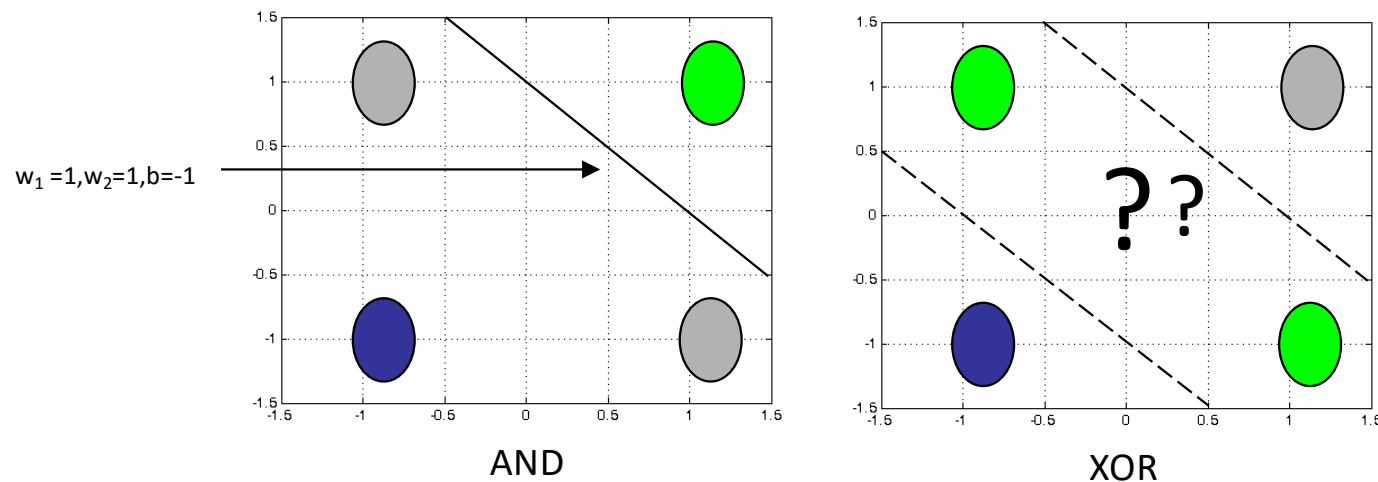
Chapter 3.3 - <https://youtu.be/QDX-1M5Nj7s?si=kBGTSSXGGukjBnms&t=1394>

Non-Linear Classification

Real-world Setting	Data Representation	Learning Paradigm	Task
	• Feature vector • Graph • Sequence	• Supervised • Unsupervised • Reinforcement	• Regression • Classification • Clustering
Model Instantiation	Model Representation	Optimization Technique	Objective Function
	Linear/ Logistic Functions (Linear Neural Networks)	Gradient Descent / Logistic Regression	Log Likelihood
	Nonlinear Neural Nets (Multilayer Perceptrons)	Gradient Descent (Back Prob)	Log Likelihood / Softmax

Linear Separability and the XOR problem

- Famous perceptron tasks: simulate simple Boolean functions (i.e. encode corresponding truth tables) by encoding inputs as -1 (for false) and +1 (for true)



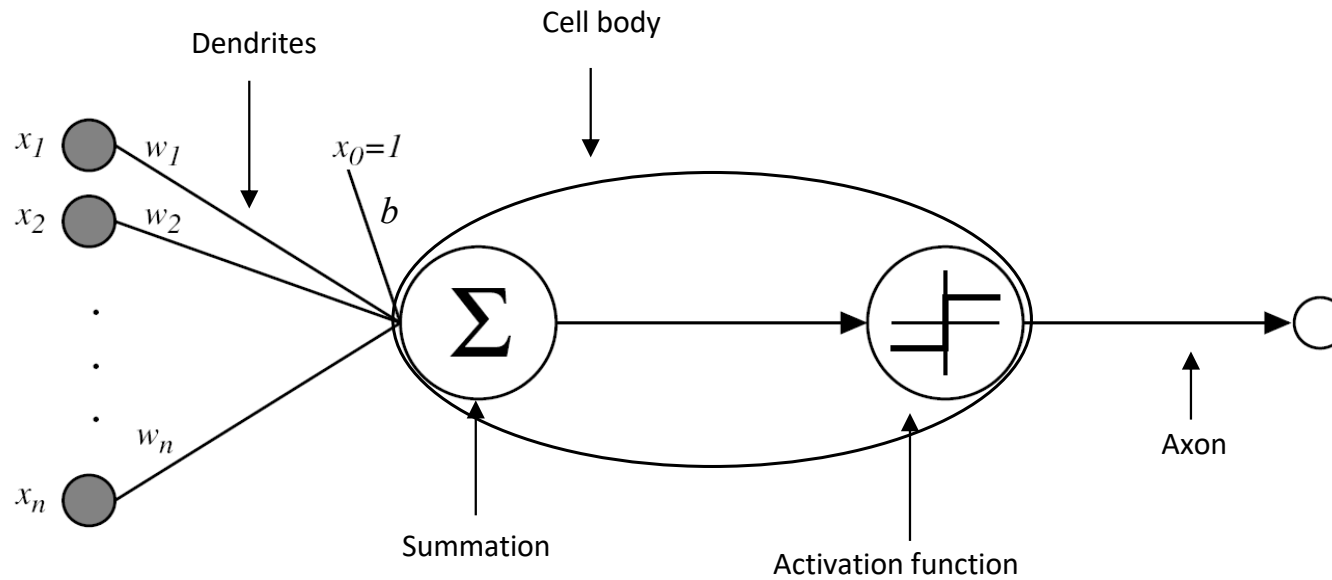
- Sad but true: even a simple XOR can not be simulated adequately by a linear threshold function.

Linear Separability and the XOR problem

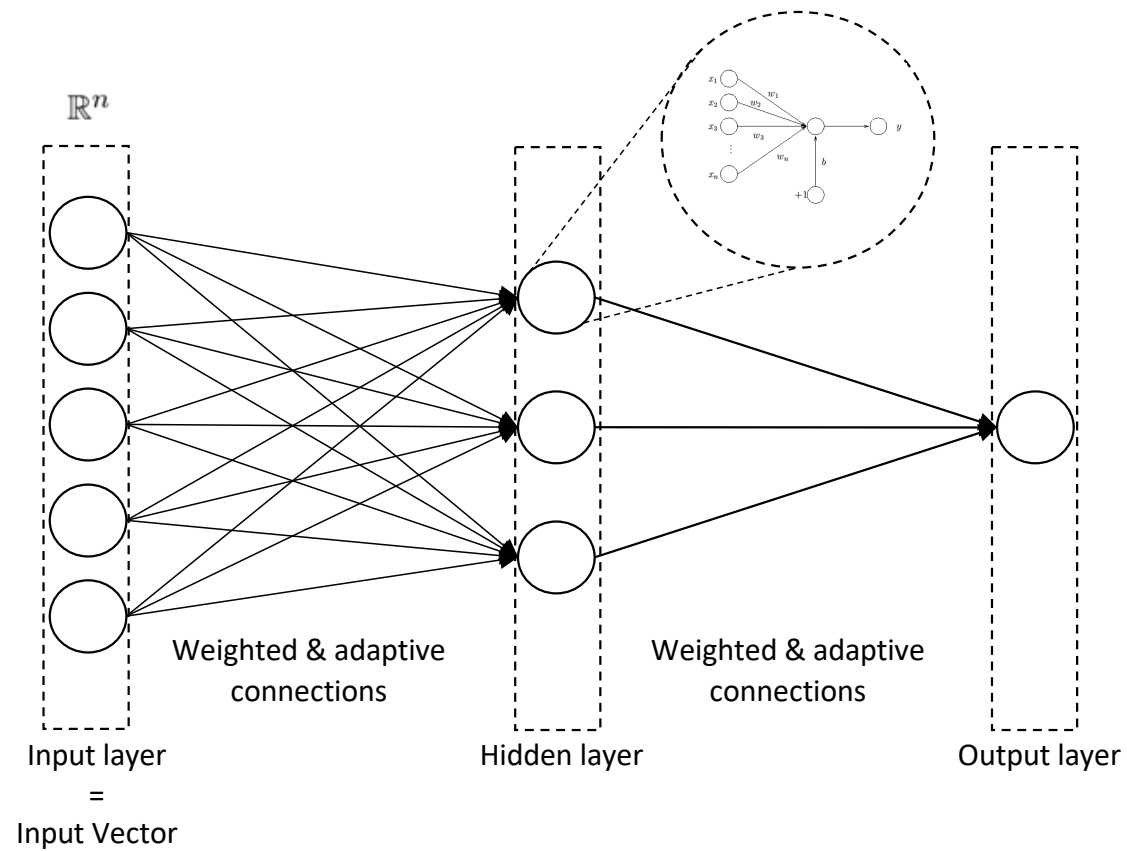
- Perceptron is able to simulate many different Boolean functions AND, OR, NAND, NOR.
 - Example: we can choose (among others) $w_1=1, w_2=1, b=-1$ for AND function.
 - NB: AND, OR are special cases of the 'm-of-n functions': at least m out of n inputs need to be 1.
- However, XOR and other more complicated expressions in propositional logic can not be expressed by a Perceptron.
- This is a **problem of the model class**, not of the training algorithm involved.

Minsky, M. and Papert, S. (1969). Perceptrons: An Introduction to Computational Geometry. MIT Press, Cambridge, MA, USA.

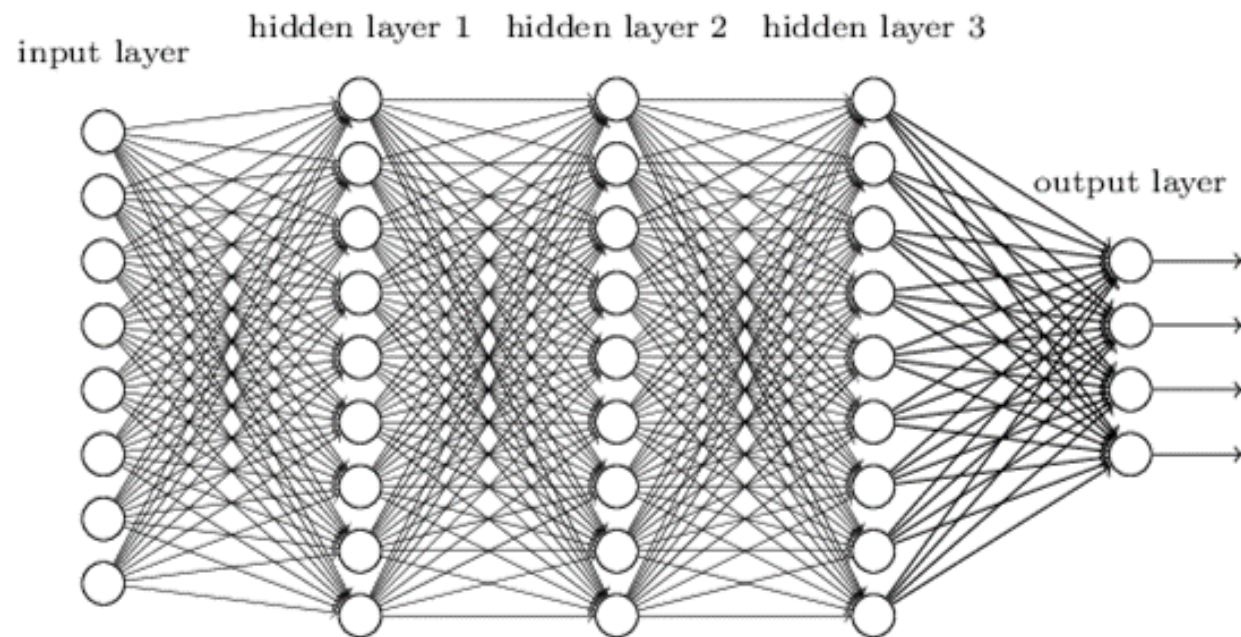
Abstract Model of the Neuron



Multilayer Perceptron



More layers



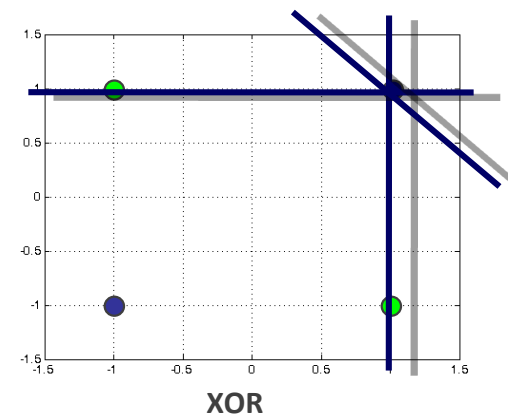
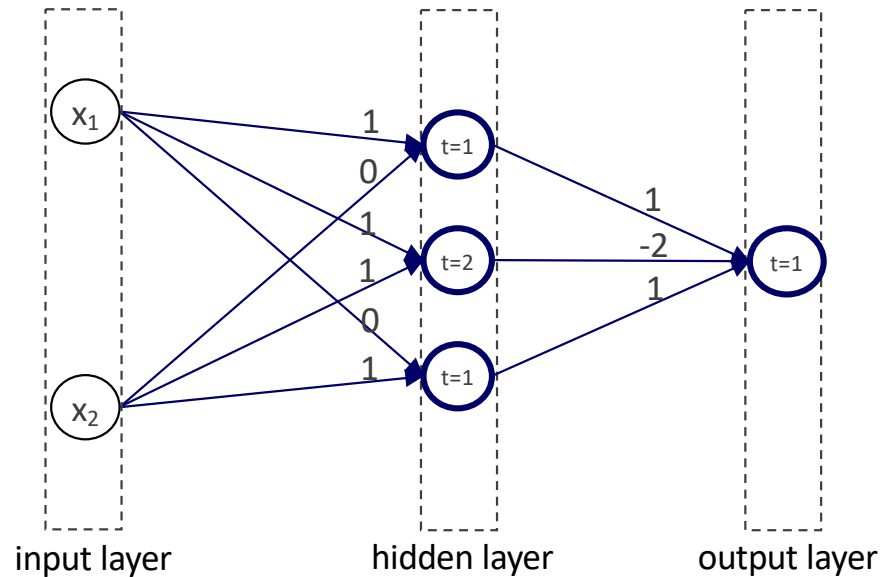
Components of an Artificial Neural Network

- **Network topology** (= model class!)
 - Set of artificial neurons
 - Structure of the connections between the neurons ("synapses")
- **Actual parameter settings** (= model)
 - Connectivity between the neurons (exhibitory vs. inhibitory)
 - Activation thresholds within the neurons
- **Training Algorithm**
 - "Learning" means determining appropriate connection and threshold weights

Multi-Layer Perceptrons

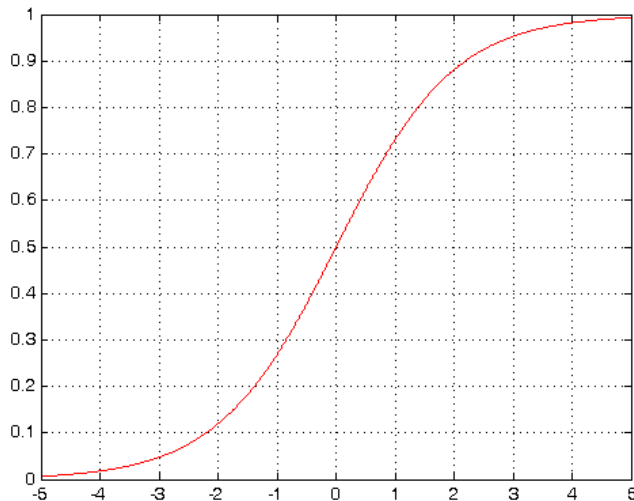
- Basic idea: combine / stack multiple neurons in multiple layers to achieve more complex prediction models.
- Activation at "hidden layers" corresponds to non-linear transformation of the input space.
- Example (next slide): two-layer neural network (don't count input layer) with one "hidden" layer (3 units) and one output layer (1 unit)

Expressivity of Multi-Layer ANNs



- t = threshold (if $\geq t$ return 1)
- XOR problem can be solved with one extra layer only

Sigmoid Activation Function



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

nice property:

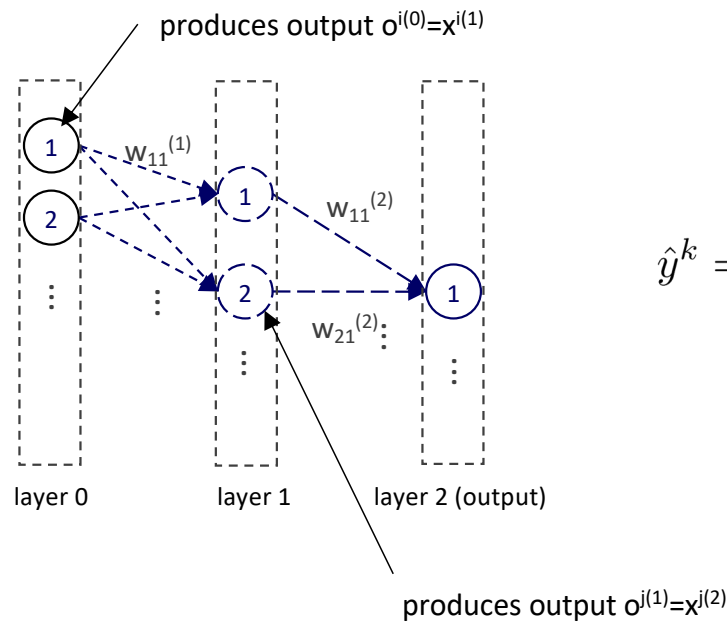
$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x))$$

- Introduces some non-linearity into the network's calculations as it "squashes" the neuron's activation into the range [0,1].
- Has an extremely simple derivative function which we need for computing gradient descent

Notation

- Network Layers 1...n; layer 0 indicates input vector
- $w_{ij}^{(l)}$ – connection weights from i -th neuron on layer $l-1$ to j -th neuron on layer l
- $x^{i(l)}$ – i -th input component to any neuron on layer l
- $net^{j(l)}$ – net input to j -th neuron (activation) on layer l with
 $net^{j(l)} = \text{Sum}_i w_{ij}^{(l)} x^{i(l)}$
- $o^{j(l)}$ – output of j -th neuron on layer l
 $o^{j(l)} = \text{Sig}(net^{j(l)}) = x^{j(l+1)}$

2-layer ANN: final activation at output layer



$$\hat{y}^k = o^{k(2)} = \sigma \left(\sum_j w_{jk}^{(2)} \sigma \left(\underbrace{\sum_i w_{ij}^{(1)} x^{i(1)}}_{net^{j(1)}} \right) \right)$$

$x^{j(2)} = o^{j(1)}$

NB: To simplify notation we again drop b parameter and include it into the weight vector: $w_0 = b$. (Imagine this to be a dummy feature that equals 1 for all inputs.)

Backpropagation – Algorithm

Input: data S ; learning rate α

Initialize: $w \leftarrow$ random values

random start values can help against local minima

1: **repeat**

2: **for** every $x \in S$ **do**

3: compute corresponding $\hat{y}$

4: **for** every unit k on output layer n **do**

5:

$$\delta_k^{(n)} \leftarrow \hat{y}^k(1 - \hat{y}^k)(y^k - \hat{y}^k)$$

6: **end for**

7: **for** every {unit k , hidden layer h } **do**

8:

$$\delta_k^{(h)} \leftarrow o^{k(h)}(1 - o^{k(h)}) \sum_{z \in \text{layer}_{h+1}} \delta_z^{(h+1)} w_{kz}^{(h+1)}$$

9: **end for**

10: update weights:

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} + \alpha \delta_j^{(l)} x_i^{(l)}$$

11: **end for**

12: **until** some termination criterion

13: **return** all $w_{ij}^{(l)}$

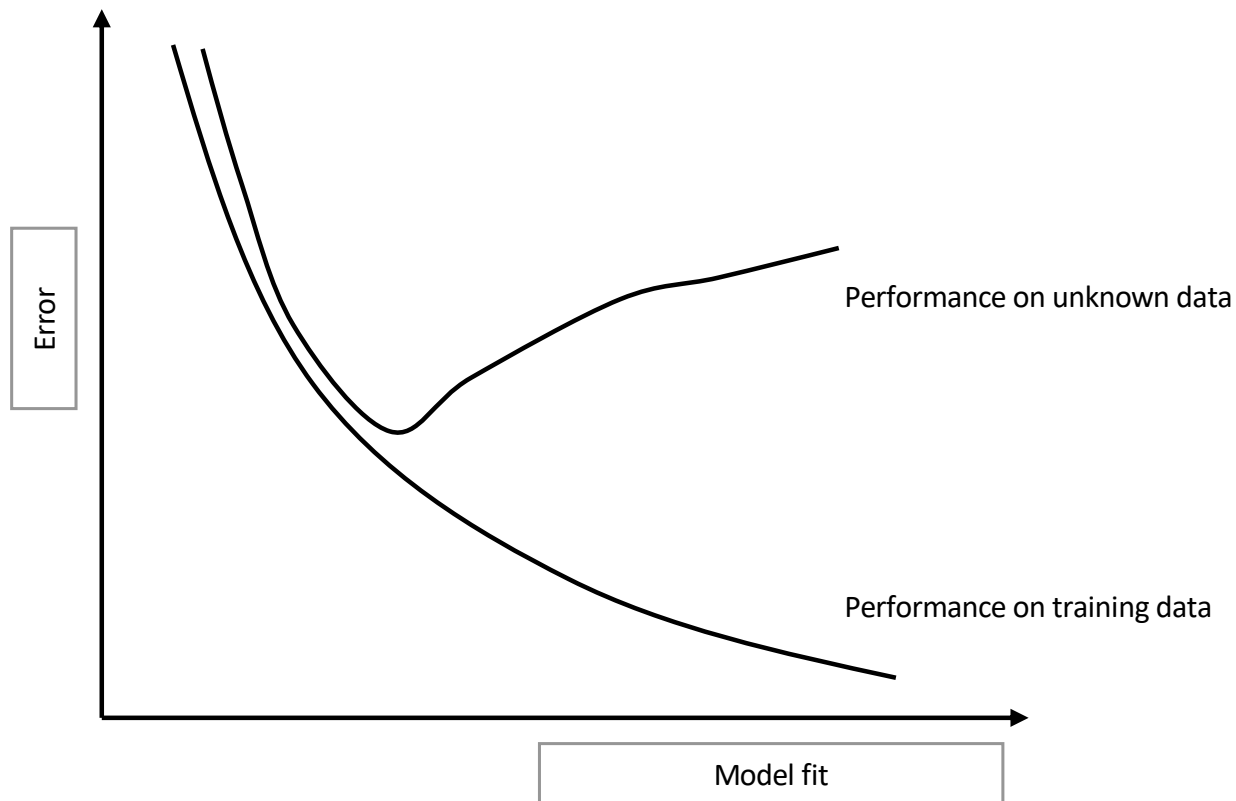
Multi-Layer ANNs – Properties

- **Universal approximation property of ANNs** - Every continuous function can be approximated arbitrarily closely by a multi-layer ANN with just one hidden layer (under further assumptions)
- **Non-Linearity** - Capable of modeling complex functions
- **Robustness** - Ignores irrelevant inputs and noise.
- **Adaptability** – it can adapt the weights based on environment changes.
- **Easy to use** – black-box view, can be used with less knowledge about relationship of the function to be modeled.

Regularization

Chapter 3.4 - <https://youtu.be/QDX-1M5Nj7s?si=hEyXApcBObzErQJ-&t=3086>

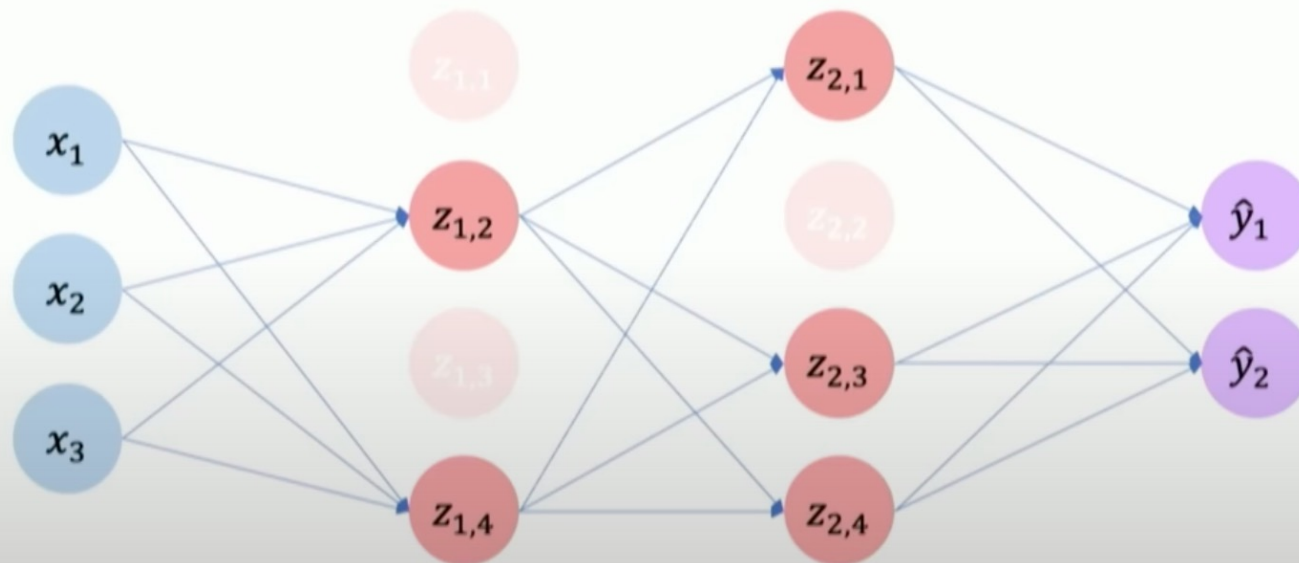
Avoiding Overfitting – Early Stopping



Avoiding Overfitting – Dropout

- During training, randomly set some activations to 0
 - Typically 'drop' 50% of activations in layer
 - Forces network to not rely on any 1 node

 `tf.keras.layers.Dropout(p=0.5)`



Avoiding Overfitting - Regularization

- regularization term / regularizer / penalty term is added to the loss

$$\text{loss}(y, \hat{y}) = \text{loss}(y, f(x, w)) + \lambda ||w||$$

“The future of language is about the development of language models...”¹

1

Computerlinguistik | Universität Trier | cl.cldh.uni-trier.de